Some kind of demo

1.

Create a simple class that represents a number and can work with it.
Essence:

- Creatw a class `Number`, that stores a private field `int value`.
- Implement:
  - a constructor with a parameter that initializes `value`;
  - methods `getValue()` and `setValue(int)` to read and modify the value;
  - a `print()` method that outputs the value to the screen with a clear label (`Number: 5`).
- In the `main()`:
  - create an object of this class;
  - set its value via the constructor or setter;
  - call the `print()` method to show how the class works.

2.

- Create a new class `NumberCollection`, that:
  - contains a private field `std::vector<Number> data`;
  - has a method `add(int v)` that creates a `Number` object with the given value and adds it to the vector;
  - has a `printAll()` method that iterates over all elements of the vector and calls `print()` for each one.
- In `main()`:
  - create a `NumberCollection` object;
  - call `add()` several times with different numbers;
  - call `printAll()` to see the output of all objects.

3.

- Create an base class `BaseNumber`, that:
  - declares the default virtual destructor;
  - declares two pure virtual functions:
    - `virtual void print() const;`
    - `virtual int getValue() const.`
- Now `Number` class should inherits from `BaseNumber`:
  - implement `print()` and `getValue()` using the `override` keyword.
- The field `value` and the constructor remain: `Number(int v = 0) : value(v) {}`.

- In the `main()`:
  - create an object of this class;
  - set its value via the constructor or setter;
  - call the `print()` method to show how the class works.

4.

- You have the base class `BaseNumber`.
- You have the first derived class `Number`.
- Add two more derived classes (three in total):
  - `SquaredNumber`
    - the constructor takes the original value `v`;
    - internally it stores, for example, `v * v`;
    - `print()` outputs that this is the square of the number;
    - `getValue()` returns the stored value.
  - `DoubleNumber`
    - the constructor takes `v`;
    - internally it stores `2 * v`;
    - `print()` shows that this is a doubled number;
    - `getValue()` returns the doubled value.
- The `NumberCollection` class should:
  - contain a vector of pointers to the base class: `std::vector<BaseNumber*> data;`;
  - have a method `add(BaseNumber* n)` that adds a pointer to the vector;
  - have a destructor that iterates over all elements and calls `delete` (because dynamic allocation is used);
  - have a `printAll()` method that iterates over the vector and calls `n->print();` — this is where polymorphism is manifested (virtual methods are called depending on the real type of the object).
- In `main()`:
  - create `NumberCollection c;`
  - add different objects:
    - `new Number(3)`
    - `new SquaredNumber(4)`
    - `new DoubleNumber(5)`
  - call `c.printAll();` to see that each object behaves differently even though all are stored as `BaseNumber*`.

5.

- Continue using the `NumberCollection` class from part four with the vector `std::vector<BaseNumber*> data;`.
- Add a method to this class, for example `int sum() const`, that:
  - creates a local variable `int s = 0;`;
  - iterates over all elements of the vector;
  - for each pointer calls `getValue()` and adds the result to `s`;
  - returns the final sum.
- In `main()` after adding the objects:
  - call `c.printAll();` to output all of them;
  - output the sum, for example:
    - `std::cout << "Sum = " << c.sum();`.